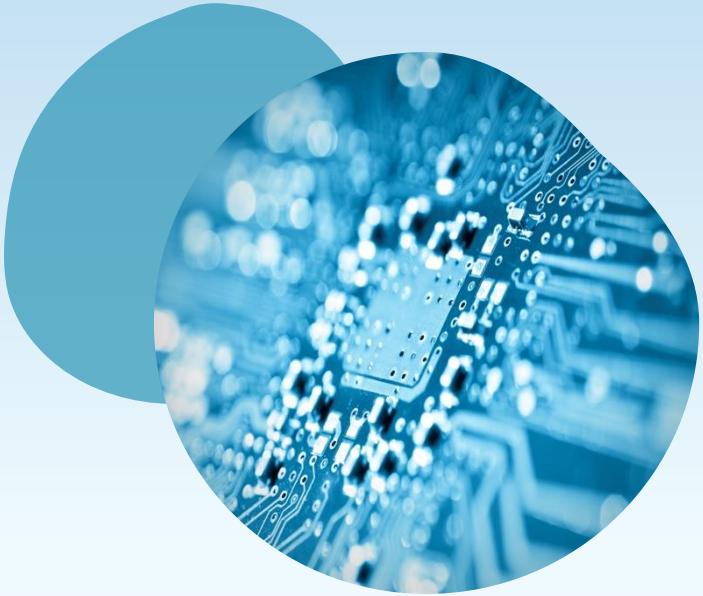
Serial Peripheral Interfaces



Embedded Software for the Internet Of Things

Kasim Sinan Yildirim

Systems Engineering: Sensor/Module Integration

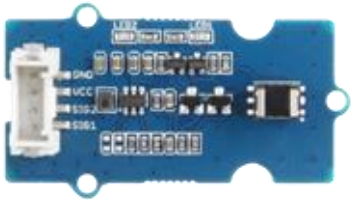
- Several Sensors, modules… How to integrate them?



Thermocouple Amp



Coulomb Counter

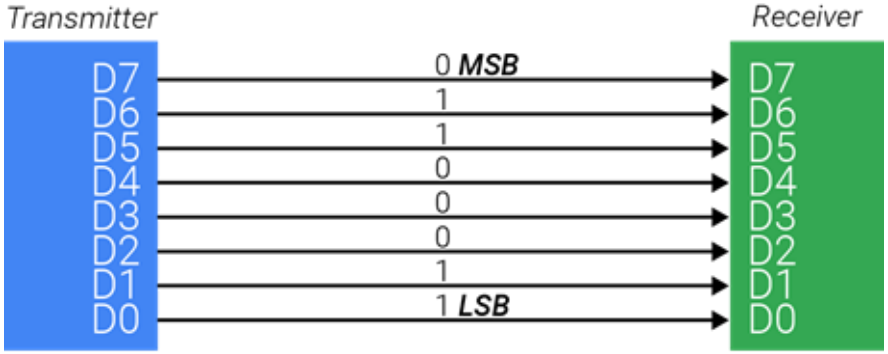


Rotary Encoder

Method of Communication

MSB → most significant bit
LSB → least significant bit

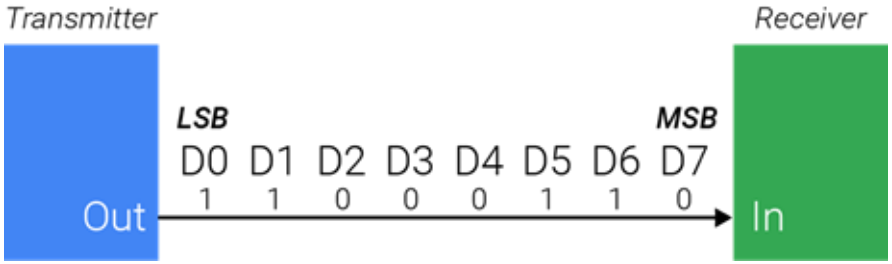
- Option 1: use **parallel interface** to connect elements



Method of Communication

MSB → most significant bit
LSB → least significant bit

- Option 2: use **serial interface** to transmit binary data in sequence



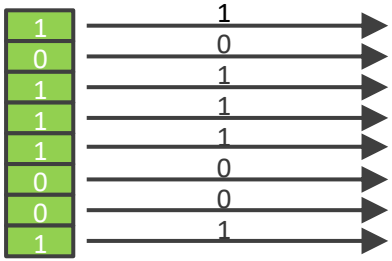
This approach **reduces number of wires**

Serial vs Parallel

- Often it's not feasible to support **sending all the bits** at the same time
  - **Cost and weight:** more wires needed, larger connectors needed
  - **Mechanical reliability:** more wires => more connector contacts to fail
  - **Timing Complexity:** some bits may arrive later than others due to variations in capacitance and resistance across conductors



Serial communication



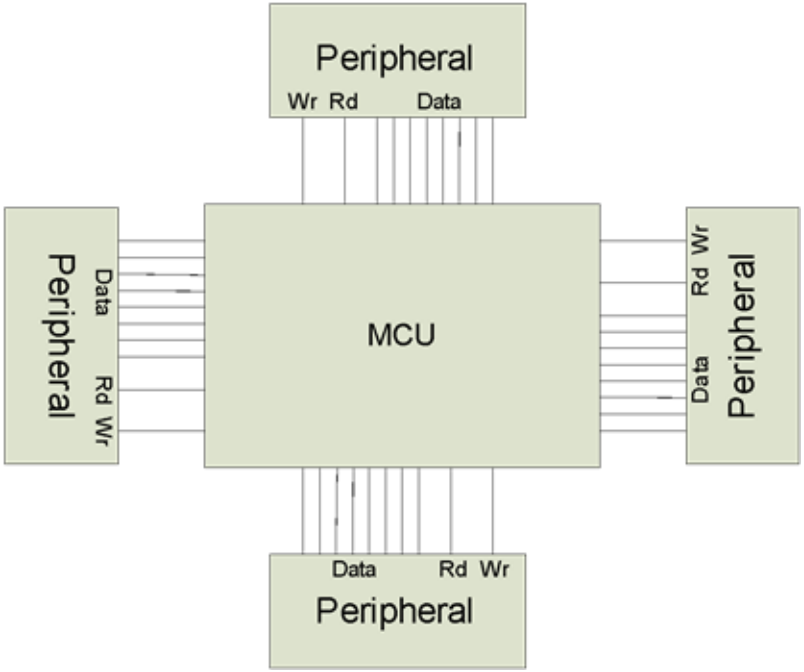
Parallel communication

Serial vs Parallel

- Cost and weight
  - Serial communication is lower cost and weight than parallel communication, as fewer wires and smaller connectors are needed.
- Serial communication is more reliable.
  - Parallel communications may introduce more clock skews, as well as crosstalk between different wires.
- Higher clock rate
  - Due to higher reliability, serial communication can be clocked at a higher frequency than parallel communication, hence increase throughput.
- On the other hand, the conversion between serial and parallel data may consume extra overheads.

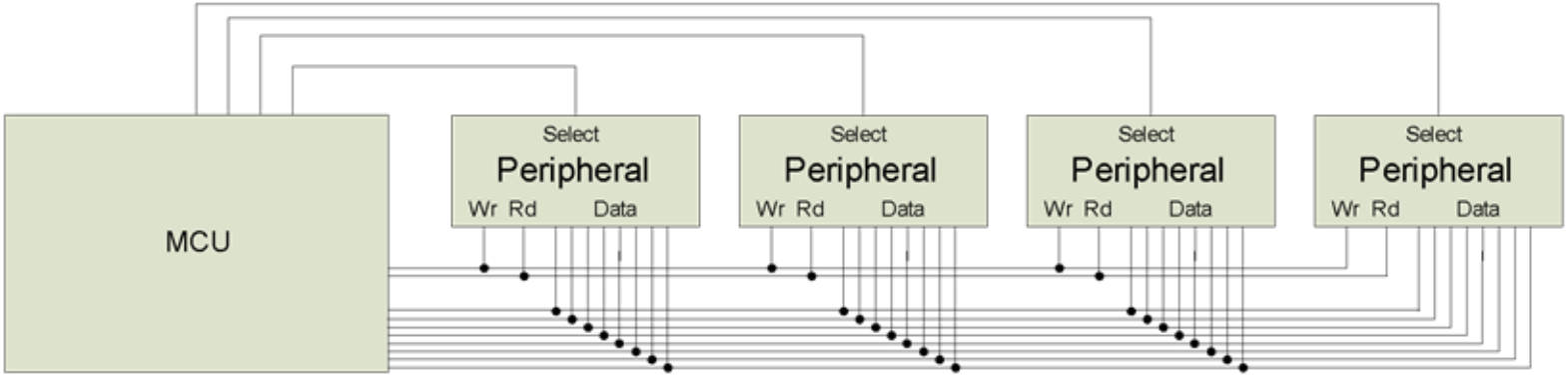
Point-to-Point Connections

- Dedicated **point-to-point** connections
- **Parallel data lines**, read and write lines between MCU and each peripheral
  - Fast, allows simultaneous transfers
  - Requires **many connections**, PCB area, scales badly



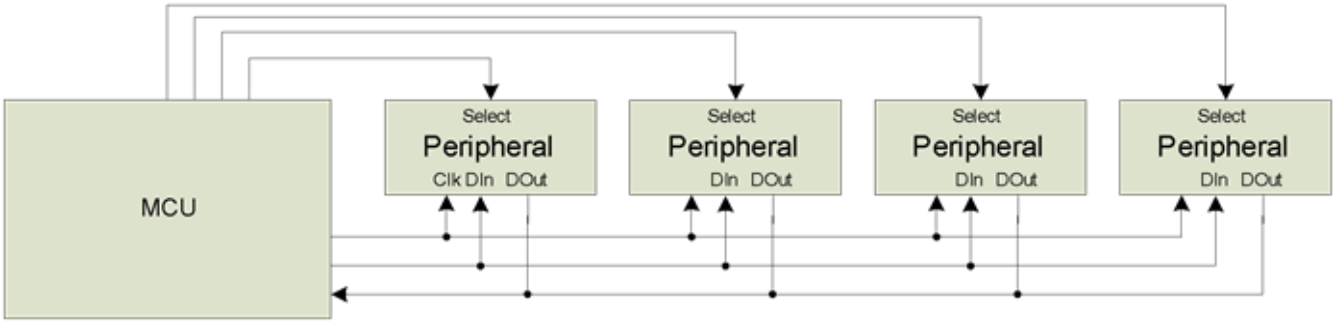
Parallel Busses

- All devices use buses to share data, read, and write signals
  - MCU uses individual select lines to address each peripheral
  - MCU requires fewer pins for data, but still one per data bit
  - MCU can communicate with only one peripheral at a time



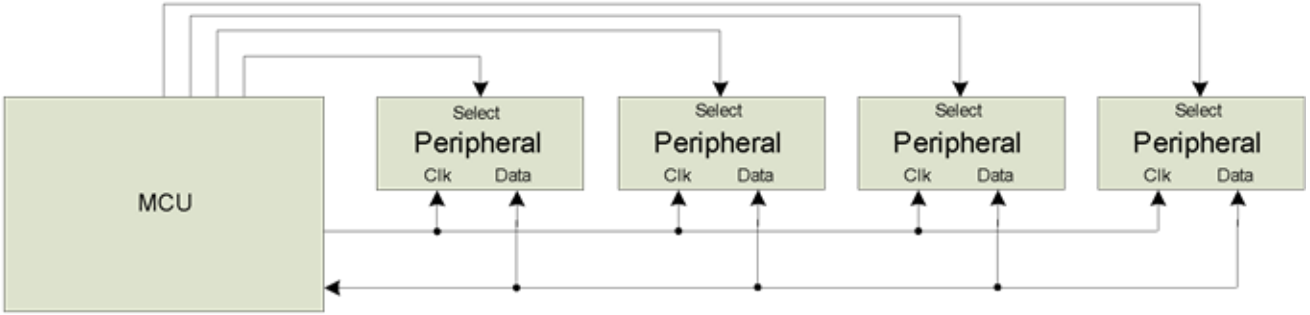
Synchronous Full-Duplex Serial Data Bus

- Now can use **two** serial data lines—one for reading, one for writing.
  - Allows **simultaneous** send and receive **full-duplex communication**



Synchronous Half-Duplex Serial Data Bus

- **Share the serial data line**
  - Doesn’t allow simultaneous send and receive
  - **half-duplex** communication



Serial Communication Protocols

UART

universal asynchronous receive-transmit

no shared clock
for synchronicity

slower

simple wiring

I2C

inter-integrated circuit

shared clock
intermediate speed

bi-directional
but one-at-a-time

simple wiring

SPI

serial peripheral interface

shared clock
high transfer speed

simultaneous
full duplex

complex wiring

Serial Communication Protocols

UART

universal asynchronous receive-transmit

no shared clock
for synchronicity

slower

simple wiring

I2C

inter-integrated circuit

shared clock
intermediate speed

bi-directional
but one-at-a-time

simple wiring

SPI

serial peripheral interface

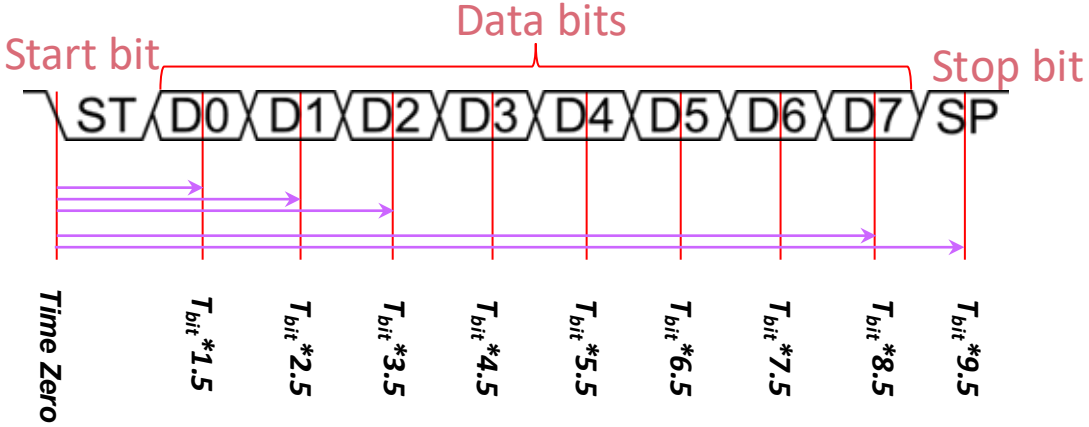
shared clock
high transfer speed

simultaneous
full duplex

complex wiring

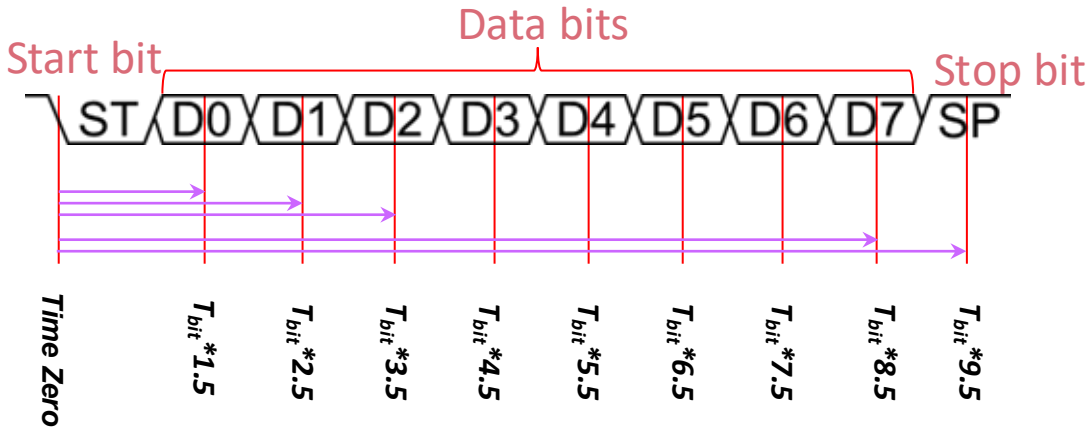
Asynchronous Serial Communication

- Eliminate the **clock line**!
  - Transmitter must add **start bit** (always same value) to indicate start of each data frame
  - Receiver detects leading edge of start bit:
    - Use it as a **timing reference** for sampling data line
  - **Stop bit** is also used to detect some timing errors



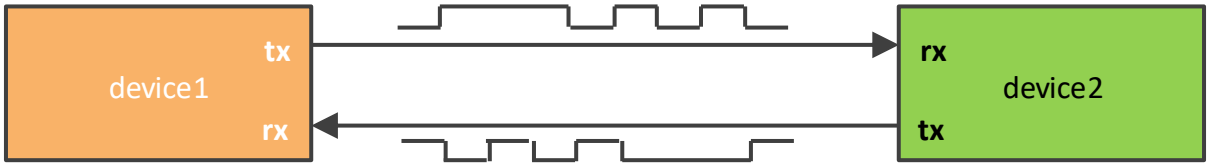
Asynchronous Serial Communication

- All devices must use the same communications parameters
  - E.g. communication speed (300 baud, 600, 1200, 2400, 9600, 14400, 19200, etc.)



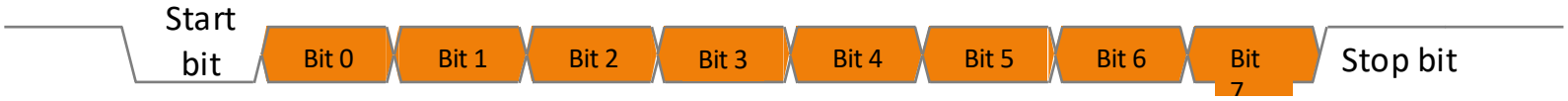
UART Overview

- **Universal Asynchronous Receiver/Transmitter (UART)**
  - Asynchronous communication, no clock wire required, pre-agreed baud rate
  - Separate transmit and receive wires
- UART communication
  - Converts data from parallel to serial
  - Sequential data is transferred through serial cable
  - Receives the sequential data and reassembles it back to parallel



UART Protocol

- Data transfer starts with a starting bit, by driving logic low for one clock cycle
- In the next 8 clock cycles, 8 bits are sent sequentially from the transmitter



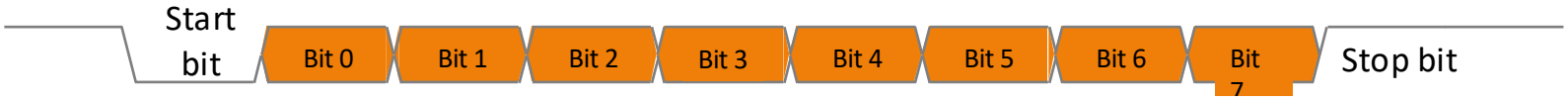
Transfer one byte without parity bit



Transfer one byte with parity bit

UART Protocol

- Optionally, 1 parity bit can be added to improve transfer reliability
- In the end, the data wire is pulled up high to indicate the end of transfer



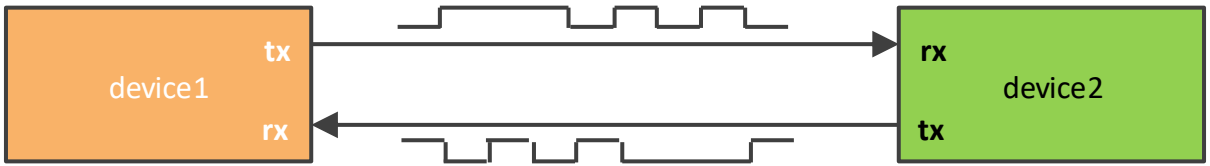
Transfer one byte without parity bit



Transfer one byte with parity bit

For this to work…

- Transmitter and receiver must agree on several things (protocol)
  - Order of data bits
  - Number of data bits
  - What a start bit is (1 or 0)
  - What a stop bit is (1 or 0)
  - How long a bit lasts
- Transmitter and receiver clocks must be reasonably close, since the only timing reference is the start of the start bit



Character-Encoding Scheme

- What is the data to be sent to the UART in order to display a text?
  - American Standard Code for Information Interchange (ASCII):
    - Encodes 128 characters
    - 95 printable characters, such as ‘a ’, ‘b’, ‘1’, ‘2’, etc.
    - 33 non-printing control characters, e.g., next line, back space, escape
    - Can be represented by 7 bits, commonly stored as one byte for storage convenience
  - UTF-8 (UCS Transformation Format—8-bit):
    - Variable-width encoding scheme, which avoids the complication of endianness and byte order marks

ASCII Encoded Characters

- The table below lists some frequently used characters coded in ASCII:

Hex	Character	Hex	Character	Hex	Character
0x30	0	0x41	A	0x61	a
0x31	1	0x42	B	0x62	b
0x32	2	0x43	C	0x63	c
0x33	3	0x44	D	0x64	d
0x34	4	0x45	E	0x65	e
0x35	5	0x46	F	0x66	f
0x36	6	0x47	G	0x67	g
0x37	7	0x48	H	0x68	h
0x38	8	0x49	I	0x69	i
0x39	9	0x4A	J	0x6A	J
…		…		…	

Error Detection

- Can send additional information to verify data was received correctly
- Need to specify which parity to expect: even, odd or none.
- Parity bit is set so that total number of “1” bits in data and parity is even (for even parity) or odd (for odd parity)
  - 01110111 has 6 “1” bits, so parity bit will be 1 for odd parity, 0 for even parity
  - 01100111 has 5 “1” bits, so parity bit will be 0 for odd parity, 1 for even parity

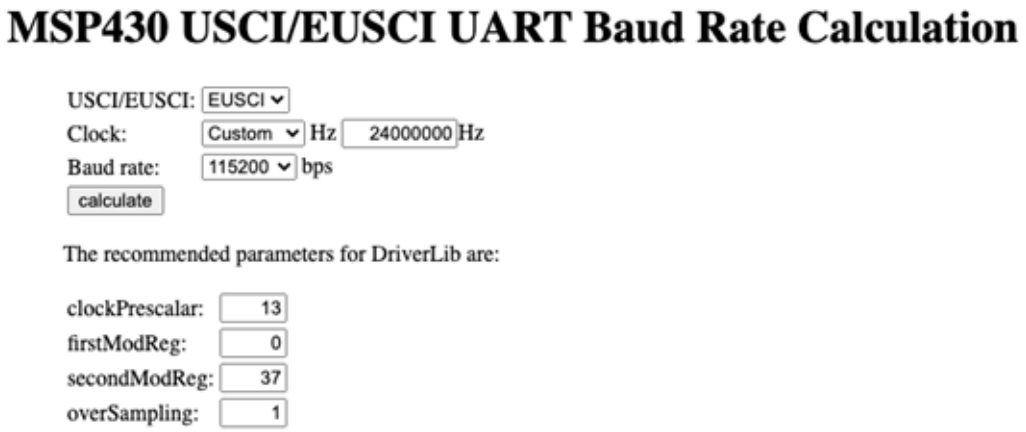
Error Detection

- Single parity bit detects if 1, 3, 5, 7 or 9 bits are corrupted, but doesn’t detect an even number of corrupted bits
- Stronger error detection codes (e.g. Cyclic Redundancy Check) exist and use multiple bits (e.g. 8, 16), and can detect many more corruptions.
  - Used for USB, Ethernet, Bluetooth, etc.

Lab1: MSP432 UART

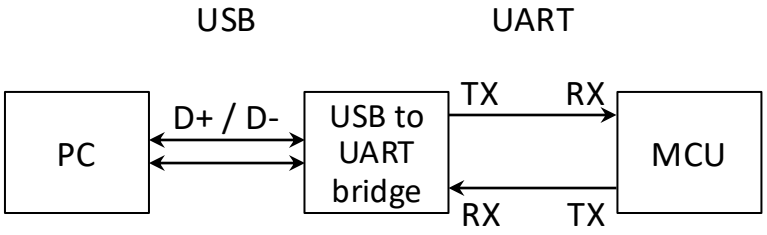
- Use the Enhanced Universal Serial Communication Interface (**eUSCI**) module to send and receive characters.

https://software-dl.ti.com/msp430/msp430_public_sw/mcu/msp430/MSP430BaudRateConverter/index.html



USB to UART Interface

- PCs do not have external asynchronous serial interfaces for a while:
  - So how do we communicate with a UART?
- USB to UART interface
  - USB connection to PC



USB01A USB to serial adapter:
http://www.pololu.com/catalog/product/391

Serial Communication Protocols

UART

universal asynchronous receive-transmit

no shared clock
for synchronicity

slower

simple wiring

I2C

inter-integrated circuit

shared clock
intermediate speed

bi-directional
but one-at-a-time

simple wiring

SPI

serial peripheral interface

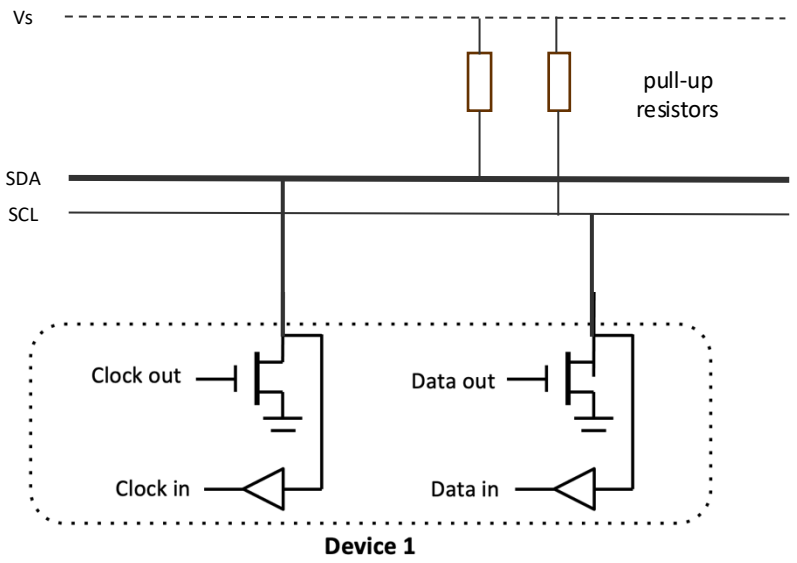
shared clock
high transfer speed

simultaneous
full duplex

complex wiring

Inter-Integrated Circuit (I2C)

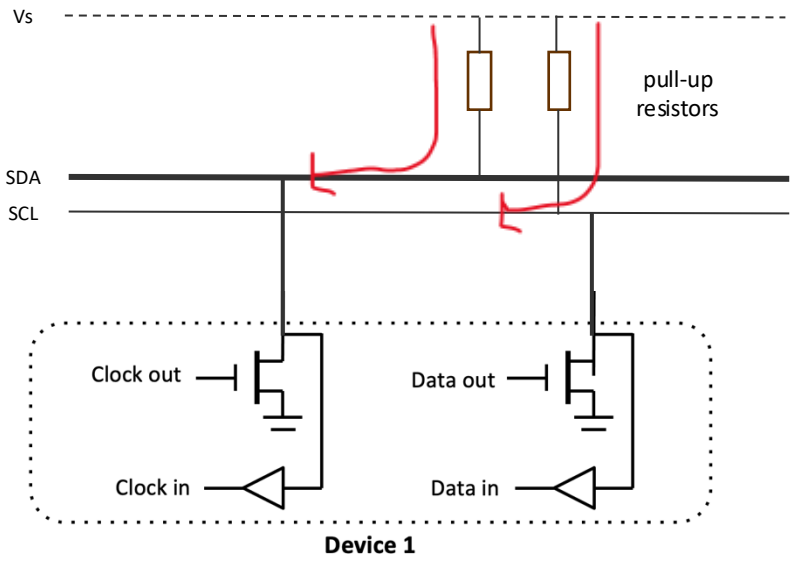
- I2C bus has two signal lines
  - **SCL:** Serial clock line
  - **SDA:** Serial data line



Inter-Integrated Circuit (I2C)

- I2C bus has two signal lines
  - **SCL:** Serial clock line
  - **SDA:** Serial data line

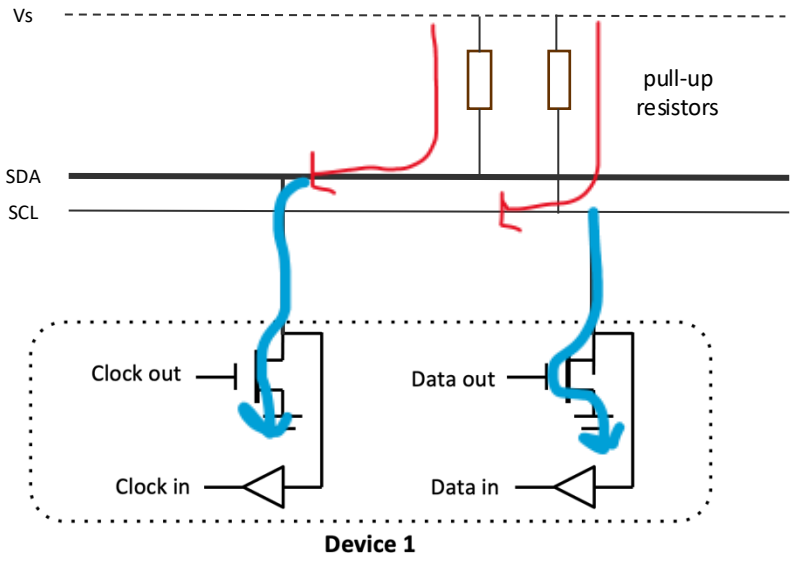
Resistors pull up lines to Vs



Inter-Integrated Circuit (I2C)

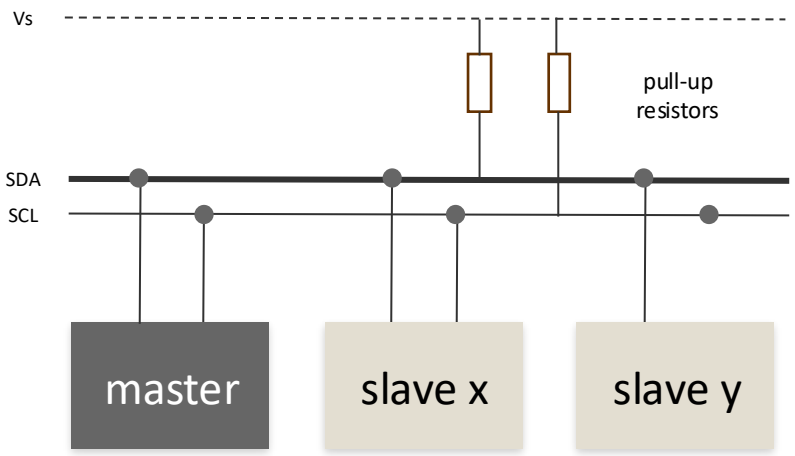
- I2C bus has two signal lines
  - **SCL:** Serial clock line
  - **SDA:** Serial data line

Resistors pull up lines to Vs
Open-drain transistors pull lines down to ground



Inter-Integrated Circuit (I2C)

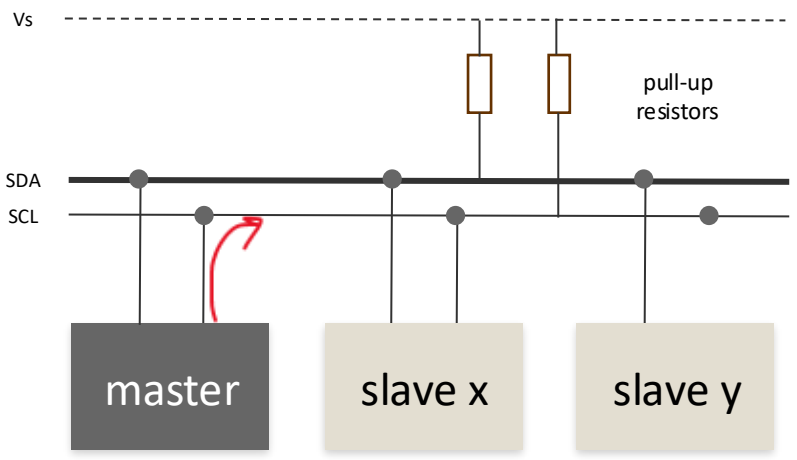
- Multiple devices connected by a shared serial bus
- Bus is typically controlled by **master device**
- Bus permits addressing of '**slaves**'
  - **Slaves** respond when addressed



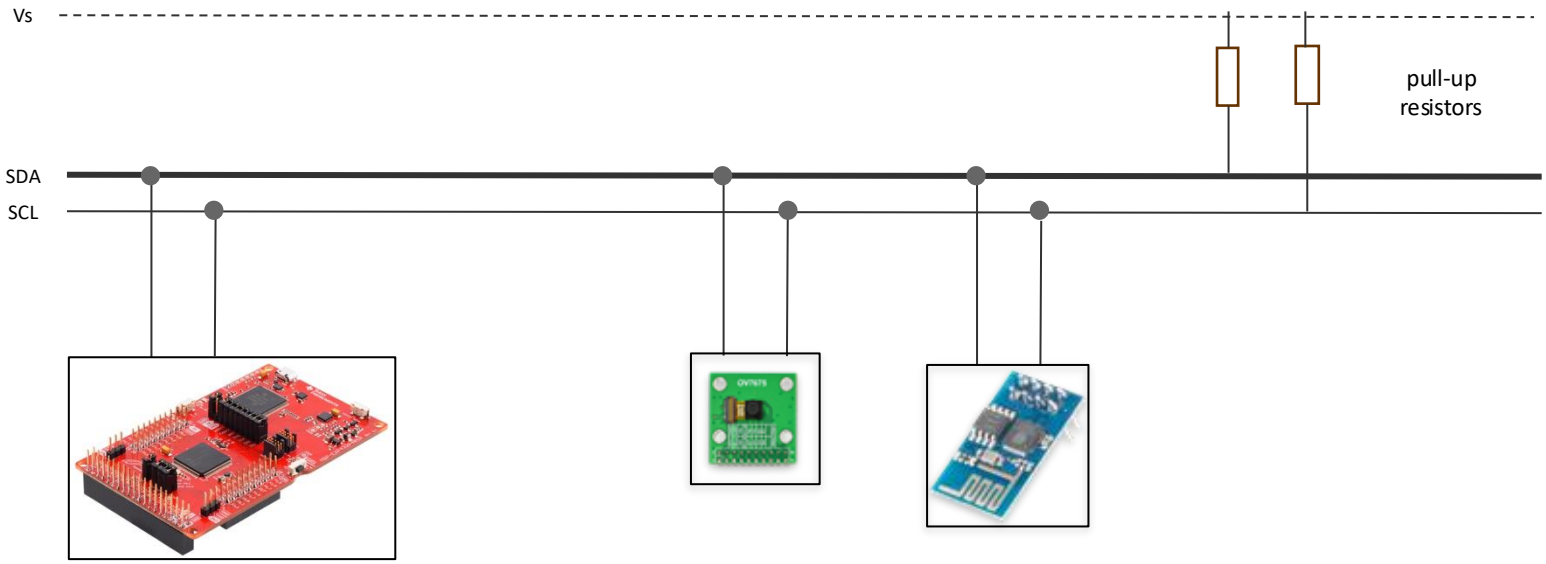
set address to slave of interest

Inter-Integrated Circuit (I2C)

- Bus masters are responsible for **generating the SCL clock**
- Slaves only respond to requests from masters.



set address to slave of interest


Vs
pull-up
resistors
SDA
SCL


START
ADDRESS
R/W
ACK
DATA
ACK
DATA
ACK
STOP


SDA
SCL
START
A1
A2
AN
STOP


START
ADDRESS
R/W
ACK
DATA
ACK
DATA
ACK
STOP


SDA
SCL
START
A1
A2
AN
STOP


START
ADDRESS
R/W
ACK
DATA
ACK
DATA
ACK
STOP


SDA
SCL
START
A1
A2
AN
STOP


START
ADDRESS
R/W
ACK
DATA
ACK
DATA
ACK
STOP


SDA
SCL
START
A1
A2
AN
STOP


START
ADDRESS
R/W
ACK
DATA
ACK
DATA
ACK
STOP


SDA
SCL
START
A1
A2
AN
STOP


START
ADDRESS
R/W
ACK
DATA
ACK
DATA
ACK
STOP


SDA
SCL
START
A1
A2
AN
STOP


START
ADDRESS
R/W
ACK
DATA
ACK
DATA
ACK
STOP


SDA
SCL
START
A1
A2
AN
STOP


START
ADDRESS
R/W
ACK
DATA
ACK
DATA
ACK
STOP


SDA
SCL
START
A1
A2
AN
STOP


START
ADDRESS
R/W
ACK
DATA
ACK
DATA
ACK
STOP


SDA
SCL
START
A1
A2
AN
STOP


START
ADDRESS
R/W
ACK
DATA
ACK
DATA
ACK
STOP


SDA
SCL
START
A1
A2
AN
STOP

I²C in action



I²C in action

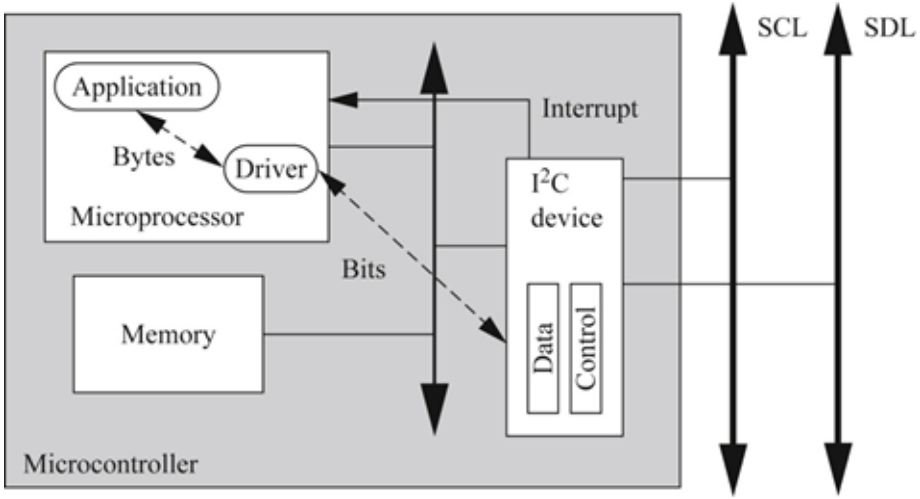


I²C in action



I2C Interface in MCU

- The I2C device takes care of:
  - generating the clock and data, acknowledges, and so forth.
- The application code calls routines:
  - to send an address, send a data byte, and so on



Serial Communication Protocols

UART

universal asynchronous receive-transmit

no shared clock
for synchronicity

slower

simple wiring

I2C

inter-integrated circuit

shared clock
intermediate speed

bi-directional
but one-at-a-time

simple wiring

SPI

serial peripheral interface

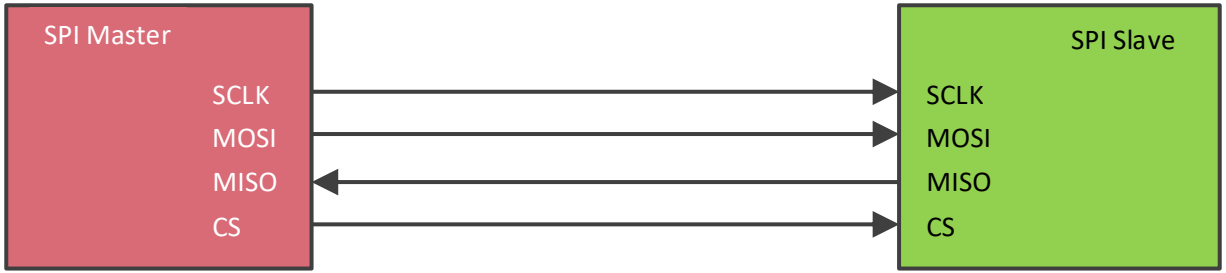
shared clock
high transfer speed

simultaneous
full duplex

complex wiring

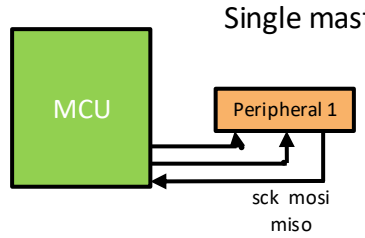
Serial Peripheral Interface (SPI)

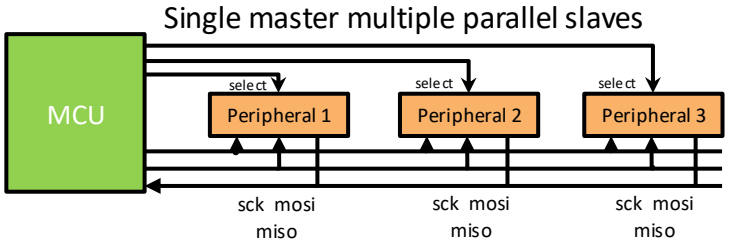
- **Four-wire** serial bus
  - Synchronous communication, clock wire required
  - Separate transmit and receive wires
  - Devices communicate in master/slave mode
- Master initiates data transfers
- Multiple slaves are enabled individually by the select lines

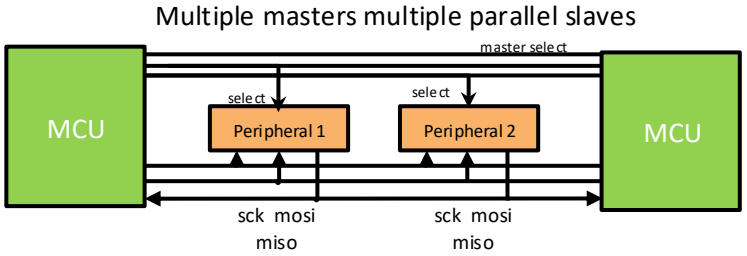


Serial Peripheral Interface (SPI)

- All chips share bus signals
  - Clock SCK
  - Data lines **MOSI** (master out, slave in) and **MISO** (master in, slave out)
- Each peripheral has its own **chip select line (CS)**
  - Master (MCU) asserts the CS line of only the peripheral it's communicating with

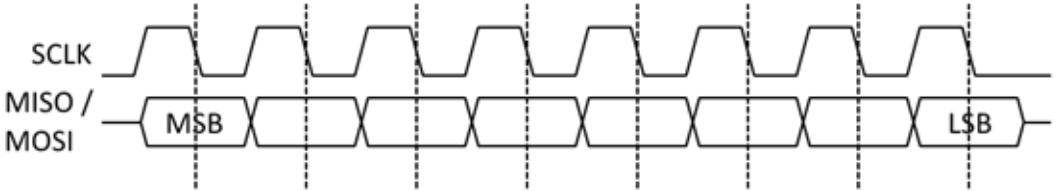






Serial Peripheral Interface (SPI)

- **Faster than I2C** and no ACK



Protocol Trade-Offs

Protocol	Speed	Signals Req. for Bidirectional Communication with N devices	Device Addressing
UART (Point to Point)	Fast – Tens of Mbit/s	2*N (TxD, RxD)	None
SPI	Fast – Tens of Mbit/s	3+N for SCLK, MOSI, MISO, and one CS per device	Hardware chip select signal per device
I^2C	Moderate – 100kbit/s, 400 kbit/s, 1Mbit/s, 3.4Mbit/s. Packet overhead.	2 (SCL, SDA)	In packet